Topic 5: Parking Lot Entry and Exit Control

Design Document

ELEC2320 – Software Fundamentals

Faculty of Engineering

Department of Electrical and Computer Engineering

University of Windsor,

Windsor, ON

GROUP 5

Bianca Junio (# 110046400)
Hashim Abu-Anzeh (#110170732)
Youssef Hendawy (# 110191147)
Denver Poulin (#110180010)
Zain Imran (#110177208)

March 1st, 2026

Introduction

This report outlines the design of a simulation system developed to help the municipality improve the operational flow of its parking facilities. Managing the movement of vehicles through entry and exit points is a complex task that requires a careful balance between the capacity of the infrastructure and the levels of staffing. Our primary goal was to create a digital environment where different management strategies could be tested and analyzed before being applied to the actual parking lot, the project structure relies on a model based on objects that represents the physical components of a parking lot, such as the gates and parking spaces, while also accounting for the behaviour of individual vehicles. By using realistic variables, such as payment methods and staff schedules, the simulation offers a clear view of how the facility handles busy periods.

Problem Statement

The municipality needs a simulation tool to determine how different staffing schedules affect the efficiency of a specific parking lot layout. Currently, congestion at both the entry and exit points poses major issues; therefore, the main objective is to determine staffing policies that are able to cut down on vehicle wait times and keep the gates running as efficiently as possible. Due to the limited facility space, the simulation must follow several realistic constraints, such as a fixed number of parking spots and parking lot containing four gates, specifically two for entry and two for exit, with each gate only able to process one car at a time. To obtain results that reflect real scenarios, the tool also needs to account for a variety of moving parts such as varying arrival rates, different payment methods, the accumulation of the queue, and duration of the parked vehicles. Overall, the simulation model must show how small changes in staff shifts can ripple through the entire system and either fix or worsen the current bottleneck.

System design

Main classes: Vehicle, Gate, Parking Lot, Simulation

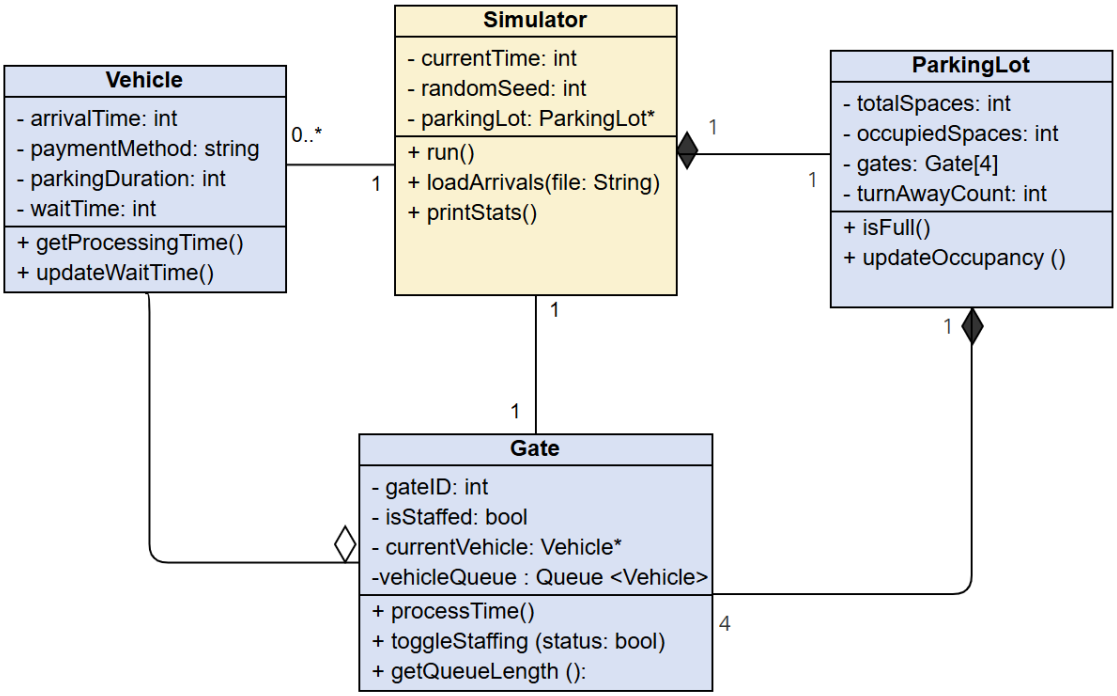
The Vehicle class stores all the specific details created in the first program, such as when the car arrives, how long it plans to stay, and how it’s going to pay. Rather than running complex logic, it mostly "records" its own experience, specifically tracking how many seconds it spends idling in line. This wait time is eventually handed back to the Simulator to help calculate the final averages.

The Gate class handles the actual processing of vehicles, which is where most of the "waiting" happens in the simulation. Each gate can only handle one car at a time and follows a strict first-come, first-served queue. The logic here is tied directly to the staffing schedule if the ‘isStaffed’status is false, the gate effectively freezes, and the queue just sits there. When it is staffed, the gate manages a timer that counts down until a vehicle is finished with its payment and can move on.

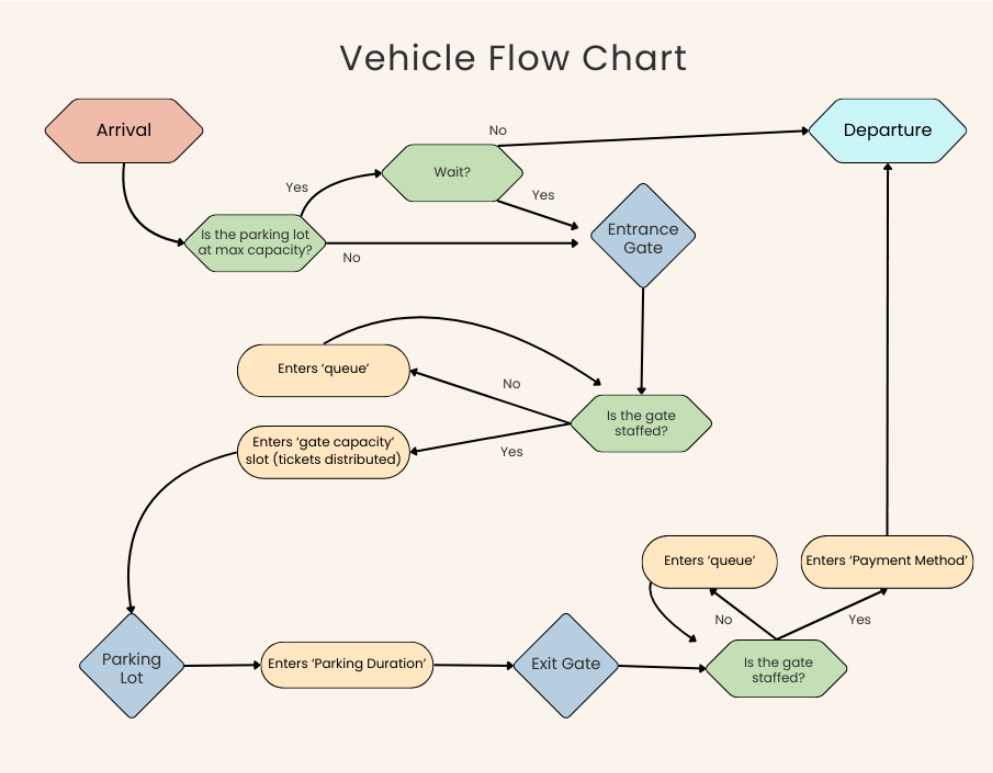
The Simulator class runs the main loop that simulates a clock and handles all the file reading and writing. Specifically, it pulls in the vehicle data from Program 1 and the staffing schedules from Program 2 to

make sure everything stays in sync. Once the simulation finishes, this class is also responsible for gathering all the raw data and turning it into the final statistics report for the municipality.

The ParkingLot class' main objective is to monitor how many cars are currently parked versus the maximum capacity of the lot. If a driver tries to enter when the lot is at its limit, this class triggers the "turn away" or "wait" policy and records that the car couldn't get in. It also serves as the parent container for the four gates, helping coordinate which gate a vehicle should head toward based on queue lengths.



(Figure 1: UML Class Diagram)



(Figure 2: Vehicle Flow Chart)

<u>Requirements</u>

Functional Requirements (including but not limited to):

- A sequence of vehicles arriving, their parking duration, and the method of payment
- Some way to staff the parking lot entry and exit include how to tell whether both the gates for entry and exit are active or inactive. This must also include the staffing schedules. The simulation should also be able to determine that an unstaffed gate will not be active and therefore no cars should be queued there.
- Since this simulation will be using more than one file, it must be able to read the files so that it runs properly
- Checking the peak times of parking usage compared to other times of the day and deciding the scheduling staff accordingly
- Ability to deal with random external factors that might affect the rest of the simulation (staff runs late, car takes too long, payment method not working, etc.)
- Ticket versus permit validation and whether prioritization to those with permits should be given. For our simulation we will give priority to those with a permit, shortening the processing time to 15s. This also includes processing times for exiting the lot.

- Parking lot capacity is important to deal with. The simulation needs to turn away cars when no more spots are available.
- From a business standpoint, the parking lot needs to calculate the average wait times, payment methods used, gate utilization, and parking lot utilization throughout the day. It must also calculate how many cars entered and left throughout the day.

Non-Functional QOL Requirements:

- The simulation must use Object Oriented Design principles when establishing the code
- The simulation needs to be built together using distinct programs so that every member of the team can individually contribute
- It needs to store a large amount of data as this data is required for other parts of the simulation to function correctly. This data should also be stored over a long time frame (1-2 days)
- The simulation and processing of the code should be efficient and not slow. A slow simulation and processing times would be detrimental to the project.
- The code needs to be easily readable with clear steps in the logic and clear defined variables so that any member of the team can easily modify the code if necessary
- The code should also be 'future-proof' where there is a clear ability to add more payment methods, staff, gates, or even parking spaces without having to rewrite the entire code.

Algorithms

Algorithm 1: Vehicle Arrival Generator (Program 1)

Purpose**:** Create a reproducible vehicle arrival file that the simulator can replay across multiple staffing schedules.

Inputs:

- Simulation duration (example: 1 day in seconds)
- Random seed (stored so results can be reproduced)
- Arrival pattern (constant rate or time blocks with different rates)
- Parking duration generation rule (fixed ranges or distributions)
- Payment method rule (cash, debit, permit)

Steps:

1. Initialize the pseudo random number generator using the provided seed.
2. Set `t = 0`.

3. While `t` is within the simulation duration:
   a. Determine the next inter arrival time using the chosen arrival pattern.
   b. Update `t = t + interArrivalTime`.
   c. Generate a vehicle record: arrival time, parking duration, and payment method.
   d. Append the vehicle record to the output list.
- Sort vehicle records by arrival time (if needed).
- Write the vehicle file in arrival order with one vehicle per line.

Algorithm 2: Gate Staffing Configuration Generator (Program 2)

Purpose: Create a staffing schedule that turns each gate “staffed” or “unstaffed” over time.

Inputs:

- Shift blocks or staffing policy rules for each of the four gates (2 entry, 2 exit)
- Simulation duration

Steps:

1. For each entry/exit gate, define a list of time intervals.
2. For each time interval, assign staffed status (ON or OFF).
3. Write a staffing file that the simulator can look up by time to update each gate’s staff state.

Algorithm 3: Main Simulation Loop (Program 3)

Purpose: Read both files and simulate vehicle flow through entry, parking, and exit while collecting statistics.

Key state tracked:

- Current simulation time `t` (discrete seconds)
- Parking lot occupancy and capacity
- Four gate queues and gate busy timers
- List of parked vehicles with their planned departure time

Service times used (As mentioned in project description):

- Ticket issuance: 25s
- Cash payment: 9 s
- Debit payment: 45s
- Permit validation: 15s

Steps (per simulated second):

1. Update staffing: For each gate, set `isStaffed` based on the staffing schedule at time `t`. Non staffed gates don't pass vehicles through
2. Add arriving vehicles to entry:
   a. For every vehicle whose arrival time equals `t`, decide whether the lot can accept it.
   b. If the lot is full, apply the chosen policy (vehicle waits or vehicle leaves) and record "turned away" if it leaves.
   c. If space is available, choose an entry gate (for example: shortest queue among staffed entry gates) and add the vehicle to the queue.
3. Move vehicles from parking to exit:
   a. For every parked vehicle whose departure time equals `t`, choose an exit gate (example: shortest queue among staffed exit gates) and add it tot he exit queue.
4. Process each gate: For each of the four gates:
   a. If gate is staffed and currently idle and its queue is not empty, service the next vehicle.
   b. Compute service time = base time (from payment method) plus a small random delay.
   c. If the gate is busy, start to decrement its remaining service timer by 1 second.
   d. When service completes:
      - If it was an entry gate, increment occupancy and place vehicle into "parked" state with a departure time = `t + parkingDuration`.
      - If it was an exit gate, decrease occupancy and mark vehicle as completed.
5. Collect stats for each tick, track queue lengths, occupancy over time, gate busy vs idle time, and per vehicle wait time.
6. Increment `t` and repeat until end of simulation time.
7. Print final report: average entry waits, average exit wait, gate utilization, vehicles turned away, and parking utilization over time.

<u>Task distribution</u>

Role	Assigned:
Architect – Defines the code structure and class hierarchy: Responsible for creating the classes and the interface of the code	Hashim
Data Lead – File input and output and generation Responsible for making sure that the data saved by the generators can be read by the simulation	Chenyiyi
Logic Dev – Build the actual simulation logic and all the main loop	Bianca

Responsible for building the core logic of the time, the random factor, how vehicles move from queue to gate etc.	
QA/Stats – Math, validation and testing Responsible for creating the associated math with the wait times amongst other validations. This also includes testing the code rigorously for errors.	Denver
Designer – User friendly interface and easy to read design Responsible for ensuring that the dashboard of the simulation is clean, easy to read with a good dashboard that shows all the information necessary about the simulation.	Zain
Integration and DevOps – Component linking and workflow Responsible for integrating the different modules of the code into one main code. Also keeps the team on track and the README file documentation.	Youssef

Conclusion

In conclusion, this project presents an object oriented simulation tool that allows the municipality to evaluate different gate controls and access control policies in a consistent and repeatable way. The system is divided into three programs, including a vehicle arrival generator, a staffing configuration generator, and the main simulation engine. This separation makes it easier to compare different scenarios under the same traffic conditions. The use of classes such as Vehicle, Gate, Parking Lot, and Simulation keeps the program organized and scalable. The overall design supports clarity, efficiency, and maintainability.

The simulation measures important performance factors such as average wait times, gate utilization, vehicles turned away, and parking space usage over time. It considers different payment methods, queue lengths, staffing schedules, and small random delays just like real world conditions. This allows the municipality to observe how congestion changes under different configurations. Overall, the project provides a practical and data driven approach to improving parking lot operations.